

HDQC-26

Question:

Write a C program that displays the following menu and uses a switch statement to print the “Right-aligned patterns” selected by the user. Use nested loops without arrays. Each pattern may be implemented in a separate parameterless function with no return value.

Task1: Algorithm:

Step 1: Start.

Step 2: Define the functions:

- increasingTriangle()
- numberTriangle()
- decreasingTriangle()

Step 3: Declare an integer variable choice.

Step 4: Display the following menu:

1. Increasing Triangle
2. Increasing Number Triangle
3. Decreasing Triangle
4. Exit

Step 5: Read the user's choice.

Step 6: Use a switch statement to perform the selected operation.

Case 1: Increasing Triangle

1. Read the number of rows n.
2. Repeat for i = 1 to n.
3. For each row, repeat j = 1 to i.
4. Print * in each iteration.
5. Move to the next line after completing each row.

Case 2: Increasing Number Triangle

1. Read the number of rows n.
2. Repeat for i = 1 to n.
3. For each row, repeat j = 1 to i.
4. Print the value of j.
5. Move to the next line after completing each row.

Case 3: Decreasing Triangle

1. Read the number of rows n.

2. Repeat for $i = n$ down to 1.
3. For each row, repeat $j = 1$ to i .
4. Print * in each iteration.
5. Move to the next line after completing each row.

Case 4: Exit

Display "Exiting program..." and terminate the menu loop.

Default Case

Display "Invalid choice!" if the user enters a choice other than 1, 2, 3, or 4.

Step 7: Repeat Steps 4–6 until the user selects **4 (Exit)**.

Step 8: Stop.

Task2: Test case table:

Test Case	Choice	n	Expected Result
TC1	1	4	Increasing * triangle with 4 rows
TC2	2	4	Increasing number triangle with 4 rows
TC3	3	4	Decreasing * triangle with 4 rows
TC4	5	—	Invalid choice!
TC5	4	—	Program exits

Task3: C Program:

```
#include <stdio.h>
void increasingTriangle();
void numberTriangle();
void decreasingTriangle();
int main()
{
    int choice;
    do
    {
        printf("\n--- PATTERNS MENU ---\n");
        printf("1. Increasing triangle\n");
        printf("2. Increasing number triangle\n");
        printf("3. Decreasing triangle\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```

switch(choice)
{
    case 1:
        increasingTriangle();
        break;
    case 2:
        numberTriangle();
        break;
    case 3:
        decreasingTriangle();
        break;
    case 4:
        printf("Exiting program...\n");
        break;
    default:
        printf("Invalid choice!\n");
}
} while(choice != 4);
return 0;
}

```

```

/* Pattern 1 */
void increasingTriangle()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= i; j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}

```

```

/* Pattern 2 */
void numberTriangle()

```

```
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= i; j++)
        {
            printf("%d ", j);

        }

        printf("\n");
    }
}
```

```
/* Pattern 3 */
void decreasingTriangle()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = n; i >= 1; i--)
    {
        for(j = 1; j <= i; j++)
        {
            printf("* ");

        }

        printf("\n");
    }
}
```

Task4: Output:

```
--- RIGHT-ALIGNED PATTERNS ---
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: 1
Enter number of rows: 4
*
* *
* * *
* * * *

--- RIGHT-ALIGNED PATTERNS ---
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: 2
Enter number of rows: 4
1
1 2
1 2 3
1 2 3 4

--- RIGHT-ALIGNED PATTERNS ---
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: 3
Enter number of rows: 4
* * * *
* * *
* *
*

* --- RIGHT-ALIGNED PATTERNS ---
```